

The Ultimate Web Development Handbook



Changing `textContent`, `innerHTML`, `value`, and `style` in JavaScript

In this section, you'll learn how to dynamically change content and appearance on a webpage using JavaScript.

1. `textContent`

The `textContent` property sets or returns the text content of a node and its descendants. It ignores any HTML tags.

```
<p id="demo">Hello <strong>World</strong></p>
```

```
const para = document.getElementById("demo");  
console.log(para.textContent); // Hello World  
para.textContent = "New text content";
```

2. `innerHTML`

The `innerHTML` property sets or returns the HTML content of an element.

```
<p id="demo">Hello</p>
```

```
const para = document.getElementById("demo");  
para.innerHTML = "<strong>Bold Text</strong>";
```

Use `innerHTML` carefully to avoid XSS vulnerabilities when inserting user-provided data.

3. `value`

The `value` property is used to get or set the value of form elements such as `<input>`, `<textarea>`, and `<select>`.

```
<input type="text" id="username" value="John">
```

```
const input = document.getElementById("username");
console.log(input.value); // John
input.value = "Harry";
```

4. `style`

You can change the inline style of an element using the `style` property.

```
<p id="demo">This is a paragraph.</p>
```

```
const para = document.getElementById("demo");
para.style.color = "red";
para.style.fontSize = "20px";
para.style.backgroundColor = "#f0f0f0";
```

Note: CSS property names with dashes (e.g., `background-color`) become camelCase (`backgroundColor`) in JavaScript.

Summary

Property	Purpose
textContent	Changes plain text (ignores HTML)
innerHTML	Changes or inserts HTML content
value	Reads or sets form input values
style	Dynamically changes inline CSS

Working with Attributes and Classes in JavaScript

JavaScript allows you to dynamically control HTML element attributes and CSS classes, enabling powerful, interactive web experiences.

1. Setting and Getting Attributes

Use `setAttribute()` to add or update an attribute and `getAttribute()` to retrieve its value.

Example:

```

```

```
const img = document.getElementById("myImage");

// Set a new attribute
img.setAttribute("alt", "Company Logo");

// Get an attribute value
console.log(img.getAttribute("src")); // "logo.png"

// Update an existing attribute
img.setAttribute("src", "new-logo.png");
```

2. Removing Attributes

Use `removeAttribute()` to completely remove an attribute from an element.

Example:

```
const img = document.getElementById("myImage");

// Remove the alt attribute
img.removeAttribute("alt");
```

3. Adding Classes

Use `classList.add()` to add one or more CSS classes to an element.

Example:

```
<div id="box"></div>
```

```
const box = document.getElementById("box");

box.classList.add("active");
box.classList.add("highlight", "shadow"); // Multiple classes
```

4. Removing Classes

Use `classList.remove()` to remove one or more classes.

```
box.classList.remove("highlight");
```

5. Toggling Classes

Use `classList.toggle()` to add a class if it doesn't exist, or remove it if it does.

```
box.classList.toggle("hidden");
```

6. Checking for a Class

Use `classList.contains()` to check whether an element has a specific class.

```
if (box.classList.contains("active")) {  
    console.log("The box is active");  
}
```

Summary

Task	Method
Set attribute	<code>element.setAttribute()</code>
Get attribute	<code>element.getAttribute()</code>
Remove attribute	<code>element.removeAttribute()</code>
Add class	<code>element.classList.add()</code>
Remove class	<code>element.classList.remove()</code>
Toggle class	<code>element.classList.toggle()</code>
Check class existence	<code>element.classList.contains()</code>

Creating, Appending, and Removing Elements in JavaScript

JavaScript allows you to dynamically create, insert, and remove elements from the DOM, enabling highly interactive web applications.

1. Creating Elements

Use `document.createElement()` to create a new HTML element in memory (not yet in the DOM).

Example:

```
const newDiv = document.createElement("div");
newDiv.textContent = "Hello, I was created dynamically!";
```

You can also set attributes or classes:

```
newDiv.setAttribute("id", "dynamicDiv");
newDiv.classList.add("box", "highlight");
```

2. Appending Elements

Use methods like `appendChild()` or `append()` to insert an element into the DOM.

Example:

```
<div id="container"></div>
```

```
const container = document.getElementById("container");  
container.appendChild(newDiv); // Adds the newDiv as the last child
```

Or using `append()` which allows multiple nodes or strings:

```
container.append("Another text node", document.createElement("hr"));
```

3. Prepending Elements

Use `prepend()` to insert an element at the beginning of the parent.

```
const heading = document.createElement("h2");  
heading.textContent = "Welcome!";  
container.prepend(heading);
```

4. Removing Elements

To remove an element, you can use:

a) `removeChild()` (from parent)

```
container.removeChild(newDiv);
```

b) `element.remove()` (directly on the element)

```
heading.remove(); // Removes the heading element directly
```

Note: `element.remove()` is widely supported but not in very old browsers.

Summary

Task	Method
Create element	<code>document.createElement()</code>
Set text content	<code>element.textContent</code>
Add attribute or class	<code>setAttribute()</code> , <code>classList.add()</code>
Append to parent	<code>appendChild()</code> , <code>append()</code>
Prepend to parent	<code>prepend()</code>
Remove from parent	<code>parent.removeChild(child)</code>
Remove element directly	<code>element.remove()</code>

Introduction to Events in JavaScript

In web development, **events** are actions or occurrences that happen in the browser, often triggered by the user. JavaScript allows us to **listen for these events** and respond with custom behavior.

Common examples of events include:

- Clicking a button
- Pressing a key
- Submitting a form
- Hovering over an element
- Scrolling the page

Why Events Matter

Events make your web pages interactive. They allow users to engage with your content dynamically, rather than just reading static information.

Types of Common Events

Here are some commonly used event types in JavaScript:

1. `click`

Triggered when an element is clicked.

```
const button = document.querySelector('button');
button.addEventListener('click', () => {
  alert('Button clicked!');
});
```

2. `keyup`

Fires when the user releases a key on the keyboard.

```
const input = document.querySelector('input');
input.addEventListener('keyup', (event) => {
  console.log(`You typed: ${event.key}`);
});
```

3. `submit`

Occurs when a form is submitted.

```
const form = document.querySelector('form');
form.addEventListener('submit', (event) => {
  event.preventDefault(); // Prevents the page from reloading
  console.log('Form submitted');
});
```

4. `mouseover` and `mouseout`

Fired when the mouse enters or leaves an element.

```
const box = document.querySelector('.box');
box.addEventListener('mouseover', () => {
  box.style.backgroundColor = 'lightblue';
});
box.addEventListener('mouseout', () => {
  box.style.backgroundColor = '';
});
```

Attaching Event Listeners

To handle events, use the `addEventListener` method:

```
element.addEventListener('eventName', callbackFunction);
```

Example:

```
document.querySelector('h1').addEventListener('click', () => {  
  console.log('Heading clicked!');  
});
```

Summary

- Events allow JavaScript to react to user interactions.
- Use `addEventListener` to attach event handlers.
- Prevent default behavior using `event.preventDefault()` if needed (e.g., in forms).
- Always test your event listeners to ensure the intended behavior works across different browsers.

Attaching Event Listeners in JavaScript

In JavaScript, you can use the `addEventListener()` method to handle user interactions like clicks, typing, mouse movements, etc.

Syntax:

```
element.addEventListener(event, callback);
```

Common Events:

- `click` – when an element is clicked
- `mouseover` – when the mouse hovers over an element
- `keydown` – when a key is pressed
- `submit` – when a form is submitted

Examples:

```
<button id="clickBtn">Click Me</button>
<input id="inputBox" placeholder="Type something" />
<form id="myForm">
  <input type="text" required />
  <button type="submit">Submit</button>
</form>

<script>
  // Click event
  document.getElementById("clickBtn").addEventListener("click", () => {
    alert("Button was clicked!");
  });

  // Keydown event
  document.getElementById("inputBox").addEventListener("keydown", (e) => {
```

```
    console.log("Key pressed:", e.key);
  });

  // Submit event
  document.getElementById("myForm").addEventListener("submit", (e) => {
    e.preventDefault();
    alert("Form submitted!");
  });
</script>
```